**EXPERIMENT 2:**

## MATLAB-Based Modelling of VSB Modulator and Demodulator for Analog Television Communication Applications

**Aim:** To model and simulate a Vestigial Sideband (VSB) modulator and demodulator using MATLAB for analogtelevision communication applications.

**Soft ware Required:** MATLAB

## Program:

```
clc;
clear;
close all;
% MATLAB-Based Modelling of VSB Modulator and Demodulator
% for Analog Television Communication Applications
%% STEP 1: Define system parameters
Fs = 100000;           % Sampling frequency (Hz)
T = 0.1;               % Simulation time (seconds)
t = 0:1/Fs:T-1/Fs;     % Time vector
N = length(t);
Ac = 10;               % Carrier amplitude
Am = 5;                % Message amplitude
fc = 10000;            % Carrier frequency (Hz)
fm = 1000;             % Message frequency (Hz)
m = Am/Ac;             % Modulation index
Bm = 3000;             % Message bandwidth (Hz)
fVestige = 500;        % Lower-sideband vestige width (Hz)
%% STEP 2: Generate message and carrier signals
message = Am*cos(2*pi*fm*t);
carrier = Ac*cos(2*pi*fc*t);
```

```
%% STEP 3: Generate conventional AM signal
AM_signal = Ac*(1 + m*cos(2*pi*fm*t)) ...
        .* cos(2*pi*fc*t);
%% STEP 4: Apply VSB filtering in frequency domain
% Frequency vector for FFT-shifted spectrum
f = (-N/2:N/2-1)*(Fs/N);AM_FFT = fftshift(fft(AM_signal));
% Pass the carrier and upper sideband,
% together with a vestige of the lower sideband.
VSB_mask = ...
  ((f >= fc-fVestige) & (f <= fc+Bm)) | ...
  ((f >= -fc-Bm) & (f <= -fc+fVestige));
VSB_FFT = AM_FFT .* VSB_mask;
% Convert the filtered spectrum back to time domain
VSB_signal = real(ifft(ifftshift(VSB_FFT)));
%% STEP 5: Coherent VSB demodulation
% Multiply by a synchronized local carrier
demodulated = VSB_signal .* (2*cos(2*pi*fc*t));
%% STEP 6: Low-pass filtering
DEMOD_FFT = fftshift(fft(demodulated));
LPF_mask = abs(f) <= Bm;
LPF_FFT = DEMOD_FFT .* LPF_mask;
baseband = real(ifft(ifftshift(LPF_FFT)));
%% STEP 7: Recover the message signal
% Remove the carrier-related DC component.
% The upper sideband is retained, while the
% 1 kHz lower sideband is rejected by the VSB filter.
% Therefore, compensate for the half-amplitude output.
recovered = (baseband - mean(baseband)) * 2/(Ac*m);
```

```
%% STEP 8: Display calculated parameters
fprintf('VSB MODULATOR AND DEMODULATOR SIMULATION\n');
fprintf('----------------------------------------\n');
fprintf('Carrier frequency       = %.2f kHz\n', fc/1000);
fprintf('Message frequency       = %.2f kHz\n', fm/1000);
fprintf('Message bandwidth       = %.2f kHz\n', Bm/1000);
fprintf('Vestigial sideband width= %.2f kHz\n', fVestige/1000);
fprintf('Modulation index        = %.2f\n', m);
fprintf('Simulation completed successfully.\n');
%% STEP 9: Plot time-domain waveforms
figure('Name','VSB Time-Domain Analysis', ...
    'NumberTitle','off');
nPlot = round(0.01*Fs);
tp = t(1:nPlot);
subplot(4,1,1);
plot(tp, message(1:nPlot), 'b', 'LineWidth', 1.2);
grid on;
title('Original Message Signal');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(4,1,2);
plot(tp, AM_signal(1:nPlot), 'r');
grid on;
title('Conventional AM Signal');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(4,1,3);
plot(tp, VSB_signal(1:nPlot), 'm');
```

```
grid on;
title('VSB Modulated Signal');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(4,1,4);
plot(tp, message(1:nPlot), 'k--', 'LineWidth', 1.2);
hold on;
plot(tp, recovered(1:nPlot), 'g', 'LineWidth', 1.2);
hold off;
grid on;
title('Original and Recovered Message Signals');
xlabel('Time (s)');
ylabel('Amplitude');
legend('Original Message','Recovered Message');
%% STEP 10: Plot frequency-domain spectra
RX_FFT = fft(AM_signal);
VSB_SPEC = fft(VSB_signal);
REC_FFT = fft(recovered);
fpos = (0:floor(N/2))*Fs/N;
RX_MAG = abs(RX_FFT(1:floor(N/2)+1))/N;
VSB_MAG = abs(VSB_SPEC(1:floor(N/2)+1))/N;
REC_MAG = abs(REC_FFT(1:floor(N/2)+1))/N;
% Double positive-frequency amplitudes except DC and Nyquist
if mod(N,2) == 0
  RX_MAG(2:end-1) = 2*RX_MAG(2:end-1);
  VSB_MAG(2:end-1) = 2*VSB_MAG(2:end-1);
  REC_MAG(2:end-1) = 2*REC_MAG(2:end-1);
```

```
else
   RX_MAG(2:end) = 2*RX_MAG(2:end);
   VSB_MAG(2:end) = 2*VSB_MAG(2:end);
   REC_MAG(2:end) = 2*REC_MAG(2:end);
end

figure('Name','VSB Frequency-Domain Analysis', ...
      'NumberTitle','off');
subplot(3,1,1);
plot(fpos/1000, RX_MAG, 'b');
grid on;
title('Conventional AM Frequency Spectrum');
xlabel('Frequency (kHz)');
ylabel('Magnitude');
xlim([0 15]);
subplot(3,1,2);
plot(fpos/1000, VSB_MAG, 'r');
grid on;
title('VSB Frequency Spectrum');
xlabel('Frequency (kHz)');
ylabel('Magnitude');
xlim([0 15]);
subplot(3,1,3);
plot(fpos/1000, REC_MAG, 'g');
grid on;
title('Recovered Message Frequency Spectrum');
xlabel('Frequency (kHz)');
ylabel('Magnitude');
```

```
xlim([0 5]);

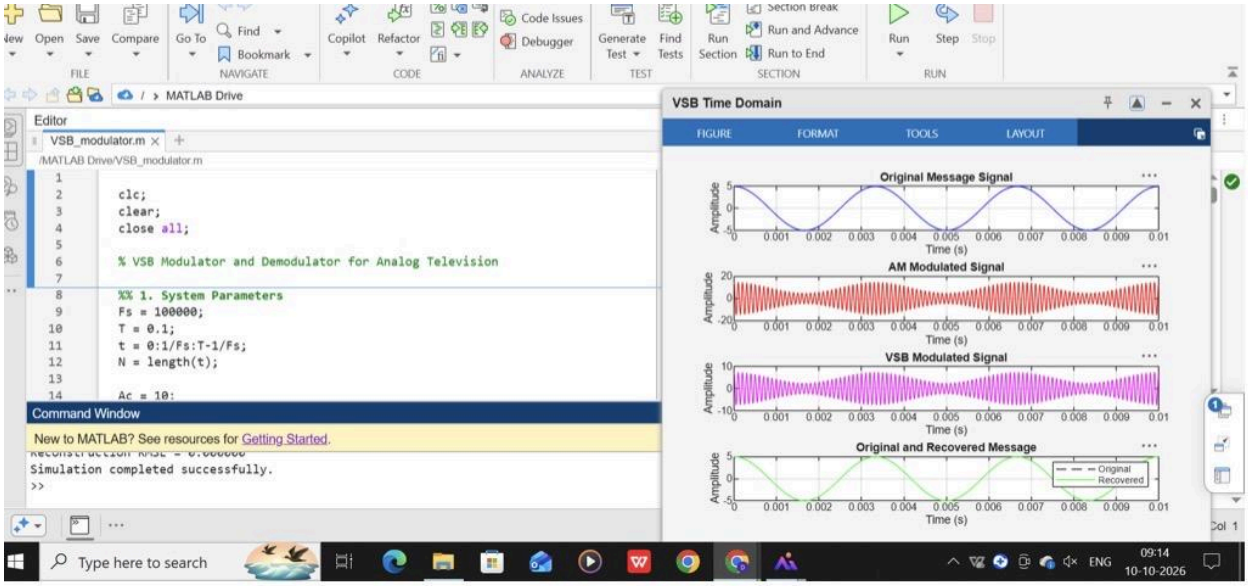
%% STEP 11: Calculate reconstruction error

error_signal = message - recovered;

RMSE = sqrt(mean(error_signal.^2));

fprintf('Reconstruction RMSE = %.4f\n', RMSE);
```

## output:




**Result:** The VSB modulator and demodulator were successfully simulated using MATLAB. The VSB frequency spectrum was analyzed, and the recovered message signal was compared with the original signal.